

Problem A. Oops, It's Yesterday Twice More

Move all kangaroos to one corner first. For example, applying $n - 1$ horizontal moves and then $m - 1$ vertical moves gathers every kangaroo at the chosen corner of the empty rectangle, regardless of its initial position.

Try all four corners and choose the one closest to the target cell. After gathering, move the group from that corner to the target along a shortest Manhattan path. The total number of moves is within the allowed bound for the closest corner.

Complexity. $O(n + m)$ output length.

Problem B. Puzzle in Inazuma

Subtract the two given complete graphs edge by edge. The problem becomes deciding whether the resulting edge weights can all be made zero.

For $n \geq 6$, the only invariant is the total sum of all edge weights. Using six vertices A, B, C, D, E, F , a fixed sequence of operations can transfer one unit from an arbitrary edge incident to one vertex to another edge, while keeping the total sum unchanged. Repeating this transfer reduces every graph with total sum zero to the zero graph.

For $n = 5$, the same construction transfers values only in steps of two, so only parities matter; enumerate the parity states of the edges. For $n = 4$, the operation variables form a small linear system with four unknowns and six equations, which can be solved directly.

Complexity. $O(n^2)$ for $n \geq 6$ after summing edges; constant-size enumeration for $n \leq 5$.

Problem C. Klee in Solitary Confinement

Enumerate the final mode value x . Only elements equal to x or $x + k$ matter: changing a chosen interval converts $x + k$ into x inside the interval, while occurrences of x outside the interval are kept.

For the positions containing either value, build a binary sequence where X means value x and Y means value $x + k$. If the chosen interval is $[l, r]$, the final number of x is

$$X_{1,l-1} + Y_{l,r} + X_{r+1,m}.$$

After rearranging, for each right endpoint r we only need the maximum prefix value of $2X_{1,l-1} - l$. Scanning the binary sequence once gives the best interval for this x . Store positions of each value in vectors so that the total scanned length over all x is linear.

Complexity. $O(n)$.

Problem D. Paimon Sorting

First assume all values are distinct. Starting from the first element, repeatedly jump to the first larger element on its right; call the visited elements special. During the first outer loop, these special elements are shifted right by one, and the maximum value moves to the front.

From the second outer loop onward, the prefix before the current position is already sorted. A non-special element contributes the number of larger values before it, while each special element contributes exactly two swaps. These contributions can be counted by a Fenwick tree.

With equal values, deduplicate the “larger before it” contribution by value. Also handle the case where a

non-special element equals the previous special element; the first shift increases its contribution by one. These are the only corner cases.

Complexity. $O(n \log n)$.

Problem E. Paimon Segment Tree

Use offline difference over time. A query on a time interval $[x, y]$ is transformed into prefix queries at times y and $x - 1$. Therefore only the current array at the current time has to be maintained.

For a segment $[l, r]$ at time t , maintain

$$x_1 = r - l + 1, \quad x_2 = \sum a_i^t, \quad x_3 = \sum (a_i^t)^2, \quad x_4 = \sum_{j=0}^t \sum (a_i^j)^2.$$

Adding k to every element changes these values by

$$x_2 \leftarrow x_2 + x_1 k, \quad x_3 \leftarrow x_3 + 2kx_2 + k^2 x_1, \quad x_4 \leftarrow x_4 + x_3 + 2kx_2 + k^2 x_1,$$

where the right-hand side uses the old x_2, x_3 . This is a linear transformation on the four maintained quantities, so it can be used as the lazy tag of a segment tree. A time step update $[l, r], k$ is split into three aligned pieces $[1, l - 1], 0, [l, r], k$, and $[r + 1, n], 0$.

The full 4×4 matrix is unnecessary in code; only the six upper-triangular coefficients that can actually change are needed.

Complexity. $O(q \log n)$.

Problem F. Paimon Polygon

There are two geometric cases: one polygon contains the other, or the two polygons are disjoint.

For the containment case, all points must lie on one side of some separating line. Build a strict convex hull A from all points, then build a strict convex hull B from the remaining points, and require that there is no point strictly inside B . Collinear triples require care: a middle point left for B can create an unwanted intersection unless there is a suitable outer point that makes the split valid.

For the disjoint case, first try to build a convex hull from all points. If this fails, the violating triple determines points that must be separated into the two sets; sweep left and right from that forced split. If the global hull succeeds, rotate around the hull and split the circular order into two consecutive angular intervals, both with angle less than 180° .

Complexity. $O(n \log n)$ for sorting/hull construction plus linear sweeps.

Problem G. Paimon's Tree

Consider first a fixed path. Suppose it contains k vertices, and the i -th path vertex can absorb b_i off-path "garbage" edges. Let

$$f(l, r, t)$$

be the best length after assigning the path edges from l to r and using the first t numbers of the sequence. The next number can extend the path to the left, extend it to the right, or be placed as garbage if the current interval still has unused garbage capacity.

For the original tree, the two endpoints of the path may still have to expand to their parents. Add a two-bit mask to the state:

$$f(u, v, t, \text{mask}),$$

where the bits say whether the left and right endpoints are still waiting to extend. The transitions are the same as in the fixed-path DP, but now the next extension follows the tree adjacency encoded by the mask. The answer is $\max f(u, v, n, 0)$.

Complexity. $O(n^3)$.

Problem H. Crystalfly

When we first enter a node u , all children of u become active. Since every lifetime satisfies $t_i \leq 3$, only two useful decisions exist:

1. enter one child and continue downward; then the other children of u cannot be collected directly from u ;
2. enter a child v , collect it, return to u , and enter another child w , which is possible only if $t_w = 3$.

Let f_u be the best value inside the subtree of u when the crystalflies at u have already disappeared before we enter u , while its children have not yet been activated. Let $\text{sum}_u = \sum_{v \in \text{ch}(u)} f_v$.

The first decision gives $\text{sum}_u + a_v$ for a chosen child v . The second gives $\text{sum}_u - f_v + a_v + \text{sum}_v + a_w$ for distinct children v, w with $t_w = 3$. Enumerate v and maintain the largest valid a_w for the second form.

The final answer is $f_1 + a_1$.

Complexity. $O(n)$.

Problem I. Cloud Retainer's Game

Without obstacles, a bouncing trajectory belongs to one of two invariant classes:

$$(x + y) \bmod 2H \quad \text{for down-right motion,} \quad (2H - y + x) \bmod 2H \quad \text{for up-right motion.}$$

Let $f(k)$ be the best number of coins obtainable on a trajectory with invariant value k .

Sweep objects in increasing x coordinate. A coin at (x, y) increases both relevant classes, because a trajectory of either direction can collect it. A removable board at (x, y) lets the two classes exchange direction, so assign both classes the maximum of their current values. Coordinate-compress the $2H$ residue classes that appear.

Complexity. $O((n + m) \log(n + m))$ with maps or sorting.

Problem J. Xingqiu's Joke

Let $\delta = a - b$. The first two operations keep δ unchanged. If we divide by a common prime factor g , then g must divide δ ; before doing so, it is optimal to move to the nearest state where both numbers are divisible by g .

Define $f(a, \delta)$ as the minimum number of operations from $(a, a + \delta)$. Enumerate each prime factor g of δ and transition to

$$f\left(\left\lfloor \frac{a}{g} \right\rfloor, \frac{\delta}{g}\right), \quad f\left(\left\lceil \frac{a}{g} \right\rceil, \frac{\delta}{g}\right),$$

with the appropriate cost to move a to the nearest multiple of g .

The second coordinate is always a divisor of the initial δ . The first coordinate has only few possible values for each divisor: applying several integer divisions in any order, with floor/ceil choices, produces at most two neighboring results. Memoized DP over these states is sufficient.

Complexity. Roughly proportional to the number of divisors of δ times the number of its prime factors.

Problem K. Ancient Magic Circle in Teyvat

Use inclusion-exclusion on each unordered tuple of four vertices. Let U be all four-tuples and let A_i be the event that the i -th of the six edges among the tuple is red. Then

$$\left| \bigcap_{i=1}^6 \overline{A_i} \right| - \left| \bigcap_{i=1}^6 A_i \right| = |U| + \sum_{k=1}^5 (-1)^k \sum_{1 \leq i_1 < \dots < i_k \leq 6} |A_{i_1} \cap \dots \cap A_{i_k}|.$$

The required value is the absolute value of this difference.

Up to isomorphism, there are only eleven possible edge patterns on four vertices. Excluding the all-red pattern, the ten remaining constrained patterns can be counted by degree ordering and adjacency lists in $O(m\sqrt{m})$. Substituting those counts into the inclusion-exclusion formula gives the answer.

Complexity. $O(m\sqrt{m})$.

Problem L. Secret of Tianqiu Valley

Solve the parity equations induced by the lamps. This tells which positions must be clicked an odd number of times and which positions must be clicked an even number of times.

If there is currently a lit lamp whose click parity must be odd, click it greedily; one move fixes one position. If no such lamp exists, there must be a currently unlit lamp that must be affected an odd number of times even though it itself needs even click parity. Then use the local four-move pattern on three consecutive useful positions, in the order 1, 2, 1, 3, to fix two positions at once.

Repeating these two steps constructs a valid sequence of length at most $2n$.

Complexity. $O(n)$.

Problem M. Windblume Festival

Each number contributes either $+a_i$ or $-a_i$ to the final expression. There must be at least one positive contribution, and when $n > 1$ there must also be at least one negative contribution. Conversely, every assignment satisfying these two conditions is realizable: choose one positive element as the starting point, merge every other positive with the nearest previous negative, and merge all remaining negatives with the starting positive.

Therefore the optimization is only over signs. If $n = 1$, the answer is a_1 . For $n > 1$, start with $\sum |a_i|$ and ensure both signs appear. If all signs would otherwise be the same, flip the element with minimum absolute value, losing $2 \min |a_i|$. Equivalently, for sorted values the answer can be written as maximum minus minimum plus the sum of all remaining absolute contributions.

Complexity. $O(n)$.